Autenticación JWT con Spring Boot 3, Spring Security 6 y MySQL

Guía práctica • Java 25 • IntelliJ IDEA • Maven • Sin Docker

Duración estimada: 6 horas (3 sesiones de 2 horas). Requisitos: JDK 25, MySQL 8, Maven e IntelliJ. Objetivo: implementar registro, autenticación JWT HS256 y un endpoint protegido con usuarios persistidos en MySQL.

1. Crear el proyecto

Spring Initializr: Maven, Java 25, Spring Boot 3.5.x, group com.ejemplo, artifact jwtmysql. Dependencias: Spring Web, Spring Security, Spring Data JPA, OAuth2 Resource Server, MySQL Driver y Validation. Evite Flyway y Docker. Si su Initializr ya no ofrece Spring Boot 3, seleccione una versión 3.5.x disponible en Maven Central. Java 25 requiere una versión de Spring Boot 3.5 compatible y un Maven moderno.

2. Crear base de datos

```
CREATE DATABASE jwt_demo CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
```

3. Configuración: src/main/resources/application.properties

```
spring.application.name=jwtmysql
spring.datasource.url=jdbc:mysql://localhost:3306/jwt_demo
spring.datasource.username=root
spring.datasource.password=${DB_PASSWORD}
spring.jpa.hibernate.ddl-auto=update
spring.jpa.open-in-view=false
spring.jpa.show-sql=true
server.port=8080
app.jwt.secret=${JWT_SECRET}
```

Configure DB_PASSWORD y JWT_SECRET en Run > Edit Configurations > Environment variables. JWT_SECRET debe contener al menos 32 bytes impredecibles; para esta implementación se recomienda una cadena ASCII aleatoria de 64 caracteres. No subir secretos a GitHub.

4. Estructura de paquetes

```
src/main/java/com/ejemplo/jwtmysql/
  JwtmysqlApplication.java
  model/AppUser.java
  repository/UserRepository.java
  security/SecurityConfig.java
  service/TokenService.java
  web/AuthController.java
  web/SecuredController.java
```

5. Entidad: model/AppUser.java

```
package com.ejemplo.jwtmysql.model;
import jakarta.persistence.*;
@Entity @Table(name="app_users")
public class AppUser {
 @Id @GeneratedValue(strategy=GenerationType.IDENTITY)
 private Long id;
 @Column(nullable=false,unique=true,length=100)
 private String username;
 @Column(nullable=false) private String password;
 protected AppUser() {}
 public AppUser(String username,String password){this.username=username;this.password=password;}
 public Long getId(){return id;}
```

```
 public String getUsername(){return username;}
 public String getPassword(){return password;}
}
```

6. Repositorio: repository/UserRepository.java

```
package com.ejemplo.jwtmysql.repository;
import com.ejemplo.jwtmysql.model.AppUser;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.Optional;
public interface UserRepository extends JpaRepository<AppUser,Long> {
 Optional<AppUser> findByUsername(String username);
 boolean existsByUsername(String username);
}
```

7. Configuración de seguridad: security/SecurityConfig.java

```
package com.ejemplo.jwtmysql.security;
import com.ejemplo.jwtmysql.repository.UserRepository;
import javax.crypto.SecretKey;
import javax.crypto.spec.SecretKeySpec;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.*;
import org.springframework.http.HttpMethod;
import org.springframework.security.authentication.*;
import org.springframework.security.config.annotation.authentication.configuration.AuthenticationConfiguration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.core.userdetails.*;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.oauth2.jose.jws.MacAlgorithm;
import org.springframework.security.oauth2.jwt.*;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.config.annotation.method.configuration.EnableMethodSecurity;
import com.nimbusds.jose.jwk.source.ImmutableSecret;
import java.nio.charset.StandardCharsets;
@Configuration @EnableMethodSecurity
public class SecurityConfig {
 @Bean PasswordEncoder passwordEncoder(){return new BCryptPasswordEncoder();}
 @Bean UserDetailsService userDetailsService(UserRepository repo){
  return username -> repo.findByUsername(username)
   .map(u -> User.withUsername(u.getUsername()).password(u.getPassword()).roles("USER").build())
   .orElseThrow(() -> new UsernameNotFoundException("Usuario no encontrado"));
 }
 @Bean AuthenticationManager authenticationManager(AuthenticationConfiguration config) throws Exception {
  return config.getAuthenticationManager();
 }
 @Bean SecretKey jwtKey(@Value("${app.jwt.secret}") String secret){
  byte[] bytes=secret.getBytes(StandardCharsets.UTF_8);
  if(bytes.length<32) throw new IllegalArgumentException("JWT_SECRET debe tener al menos 32 bytes");
  return new SecretKeySpec(bytes,"HmacSHA256");
 }
 @Bean JwtEncoder jwtEncoder(SecretKey key){return new NimbusJwtEncoder(new ImmutableSecret<>(key));}
 @Bean JwtDecoder jwtDecoder(SecretKey key){
  return NimbusJwtDecoder.withSecretKey(key).macAlgorithm(MacAlgorithm.HS256).build();
 }
 @Bean SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
  return http.csrf(csrf -> csrf.disable())
   .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
   .authorizeHttpRequests(a -> a
    .requestMatchers(HttpMethod.POST,"/api/auth/register","/api/auth/token").permitAll()
    .anyRequest().authenticated())
   .oauth2ResourceServer(o -> o.jwt(j -> {}))
   .build();
 }
}
```

Nota: desactivar CSRF es apropiado para esta API de laboratorio que autentica exclusivamente mediante Authorization: Bearer; no extrapolar la decisión a aplicaciones basadas en cookies/sesiones.

8. Servicio de tokens: service/TokenService.java

```
package com.ejemplo.jwtmysql.service;
import java.time.Instant;
import org.springframework.security.oauth2.jose.jws.MacAlgorithm;
import org.springframework.security.oauth2.jwt.*;
import org.springframework.stereotype.Service;
@Service
public class TokenService {
 private final JwtEncoder encoder;
 public TokenService(JwtEncoder encoder){this.encoder=encoder;}
 public String issue(String username){
  Instant now=Instant.now();
  JwtClaimsSet claims=JwtClaimsSet.builder().issuer("jwtmysql-lab")
   .issuedAt(now).expiresAt(now.plusSeconds(1800)).subject(username).build();
  JwsHeader header=JwsHeader.with(MacAlgorithm.HS256).build();
  return encoder.encode(JwtEncoderParameters.from(header,claims)).getTokenValue();
 }
}
```

9. Controlador: web/AuthController.java

```
package com.ejemplo.jwtmysql.web;
import com.ejemplo.jwtmysql.model.AppUser;
import com.ejemplo.jwtmysql.repository.UserRepository;
import com.ejemplo.jwtmysql.service.TokenService;
import jakarta.validation.Valid;
import jakarta.validation.constraints.*;
import org.springframework.http.*;
import org.springframework.security.authentication.*;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.web.bind.annotation.*;
import java.util.Map;
@RestController @RequestMapping("/api/auth")
public class AuthController {
 private final UserRepository repo;
 private final PasswordEncoder encoder;
 private final AuthenticationManager auth;
 private final TokenService tokens;
 public AuthController(UserRepository repo,PasswordEncoder encoder,AuthenticationManager auth,TokenService
tokens){
  this.repo=repo;this.encoder=encoder;this.auth=auth;this.tokens=tokens;
 }
 public record Credentials(@NotBlank String username,@NotBlank String password) {}
 public record Registration(@NotBlank @Size(min=3,max=100) String username,
  @NotBlank @Size(min=8) String password) {}
 @PostMapping("/register")
 public ResponseEntity<?> register(@Valid @RequestBody Registration input){
  if(repo.existsByUsername(input.username())) return ResponseEntity.status(409).body(Map.of("error","Usuario
existente"));
  repo.save(new AppUser(input.username(),encoder.encode(input.password())));
  return ResponseEntity.status(201).body(Map.of("message","Usuario registrado"));
 }
 @PostMapping("/token")
 public Map<String,String> token(@Valid @RequestBody Credentials input){
  auth.authenticate(new UsernamePasswordAuthenticationToken(input.username(),input.password()));
  return Map.of("access_token",tokens.issue(input.username()),"token_type","Bearer");
 }
}
```

Las credenciales incorrectas son rechazadas por AuthenticationManager. Para una API de producción conviene añadir un @RestControllerAdvice que estandarice errores, límites de intentos, validación de usuarios y política de contraseñas.

10. Endpoint protegido: web/SecuredController.java

```
package com.ejemplo.jwtmysql.web;
import java.util.Map;
import org.springframework.security.core.Authentication;
import org.springframework.web.bind.annotation.*;
@RestController
public class SecuredController {
 @GetMapping("/api/secured")
 public Map<String,String> secured(Authentication auth){
  return Map.of("message","Acceso autorizado","username",auth.getName());
 }
}
```

11. Pruebas con IntelliJ HTTP Client: requests.http

```
### Registrar (esperado: 201)
POST http://localhost:8080/api/auth/register
Content-Type: application/json

{"username":"estudiante1","password":"ClaveSegura123!"}

### Autenticar (esperado: 200)
POST http://localhost:8080/api/auth/token
Content-Type: application/json

{"username":"estudiante1","password":"ClaveSegura123!"}

### Sin token (esperado: 401)
GET http://localhost:8080/api/secured

### Con token (reemplazar valor; esperado: 200)
GET http://localhost:8080/api/secured
Authorization: Bearer PEGAR_TOKEN_AQUI
```

12. Actividades de verificación

☐ Consultar en MySQL: SELECT id,username,password FROM app_users; explicar por qué no aparece la contraseña original.

☐ Registrar nuevamente el mismo username y comprobar HTTP 409.

☐ Enviar contraseña incorrecta y verificar rechazo de autenticación.

☐ Acceder a /api/secured sin token y con token válido; comparar respuestas.

☐ Alterar un carácter del JWT y verificar HTTP 401.

☐ Esperar a la expiración (30 minutos) o reducirla temporalmente a 60 segundos para probar HTTP 401.

☐ Explicar por qué un usuario registrado no recibe automáticamente privilegios ADMIN.

13. Preguntas de análisis

• ¿Qué diferencia existe entre autenticación y autorización?

• ¿Qué contiene un JWT y qué parte protege su firma?

• ¿Por qué BCrypt no debe reemplazarse por SHA-256 directo para contraseñas?

• ¿Qué implica SessionCreationPolicy.STATELESS?

• ¿Por qué un token firmado no significa que sus datos estén cifrados?

• ¿Qué riesgos introduce almacenar el JWT en el navegador?

14. Evaluación sugerida

Configuración Spring Boot y conexión MySQL: 20 %

Persistencia de usuarios y BCrypt: 20 %

Autenticación y generación de JWT: 25 %

Protección y pruebas de endpoints: 25 %

Análisis conceptual y evidencias: 10 %

15. Alcance y buenas prácticas

Este ejemplo es pedagógico, no un sistema de identidad listo para producción. No incluye refresh tokens, revocación, auditoría, rate limiting, bloqueo por intentos, HTTPS, gestión centralizada de claves ni autorización granular. El endpoint /register es público solo para la práctica. La autorización 403 debe enseñarse como extensión posterior incorporando autoridades y restricciones de acceso; el flujo básico presentado permite comprobar principalmente 200, 201, 401 y 409.

Referencia: evoila, “Spring Boot 3 & JWT Authentication: Leveraging Spring Security’s Built-in Support”, https://evoila.com/gb/spring-boot-3-jwt-authentication-leveraging-spring-securitys-support/